

Trabalho Prático 03 – Hierarquia de Memória: Memória Cache

Mateus Henrique Machado Braz¹

¹Ciência da Computação – Campus UFV-Florestal
Universidade Federal de Viçosa (UFV)

mateus.braz@ufv.br
<https://github.com/MateusHMB/TP-3---OC>

Resumo. Este relatório apresenta a análise do impacto da hierarquia de memória cache no desempenho de algoritmos de ordenação e recursivos. Foram implementados e avaliados quatro algoritmos de ordenação – Bubblesort, Radixsort, Quicksort e Mergesort – além do algoritmo Torre de Hanoi, utilizando as ferramentas *perf* e *valgrind/cachegrind*. Os resultados demonstram como o tamanho da entrada e as características da cache influenciam diretamente o número de cache misses e o tempo de execução. Adicionalmente, foi desenvolvida uma versão iterativa cache-friendly da Torre de Hanoi para demonstrar técnicas de otimização de acesso à memória.

1. Organização da Cache do Processador

O processador utilizado nos experimentos é o **Intel Core i5-13420H** (13ª geração, arquitetura Raptor Lake-H), lançado no primeiro trimestre de 2023. Este processador possui arquitetura híbrida com 4 núcleos de desempenho (P-cores, Golden Cove) e 4 núcleos de eficiência (E-cores, Gracemont), totalizando 8 núcleos e 12 threads.

A hierarquia de cache é organizada conforme a Tabela 1.

Tabela 1. Organização da cache do Intel Core i5-13420H

Nível	Tipo	Núcleos	Tamanho	Assoc.	Linha
L1 Dados	P-Core	por núcleo	48 KB	12-way	64 B
L1 Instrução	P-Core	por núcleo	32 KB	8-way	64 B
L2	P-Core	por núcleo	1,25 MB	10-way	64 B
L1 Dados	E-Core	por núcleo	32 KB	8-way	64 B
L1 Instrução	E-Core	por núcleo	64 KB	8-way	64 B
L2	E-Core	compartilhado (4)	2 MB	16-way	64 B
L3 (Smart Cache)	Todos	compartilhado	12 MB	8-way	64 B

Fonte: CPU-World e CPU-Z Validator ([valid.x86.fr](https://www.cpuid.com/valid.htm)).

2. Algoritmos de Ordenação

2.1. Bubblesort

O Bubblesort é um algoritmo de ordenação simples baseado em comparações adjacentes. Em cada passagem pelo array, elementos consecutivos são comparados e trocados se estiverem fora de ordem. O processo se repete até que nenhuma troca seja necessária. Sua complexidade é $O(n^2)$ no pior e médio caso, e $O(n)$ no melhor caso (array já ordenado). Do ponto de vista de cache, o acesso é sequencial, favorecendo a localidade espacial, porém o grande número de iterações causa muitos acessos repetidos.

2.2. Radixsort

O Radixsort é um algoritmo de ordenação não-comparativo que processa os números dígito a dígito, do menos significativo ao mais significativo (LSD – *Least Significant Digit*). Para cada dígito, utiliza um Counting Sort como sub-rotina estável. Sua complexidade é $O(nk)$, onde k é o número de dígitos. Por trabalhar com arrays auxiliares e múltiplas passagens, pode gerar mais cache misses que algoritmos in-place, mas escala muito bem com n .

2.3. Quicksort

O Quicksort é um algoritmo de divisão e conquista que seleciona um pivô, particiona o array em elementos menores e maiores que o pivô, e aplica recursivamente o mesmo procedimento às duas partições. Sua complexidade média é $O(n \log n)$, com pior caso $O(n^2)$ quando o pivô escolhido é sempre o maior ou menor elemento. Por operar in-place e ter bom comportamento de localidade de cache, é geralmente o mais eficiente na prática.

2.4. Mergesort

O Mergesort é um algoritmo de divisão e conquista que divide o array ao meio recursivamente até obter subarrays de tamanho 1, e então os mescla (*merge*) de forma ordenada. Sua complexidade é $O(n \log n)$ em todos os casos – melhor, médio e pior – o que o torna previsível e estável. A desvantagem é que requer $O(n)$ de memória auxiliar para o processo de mesclagem. Por acessar arrays de forma sequencial durante o merge, apresenta boa localidade espacial.

3. Experimentos com perf – Itens 3 e 4

Os algoritmos foram compilados com `gcc -O0 -g` (sem otimizações) e executados com a ferramenta `perf stat` monitorando os eventos: `cache-references`, `cache-misses`, `task-clock`, `cycles` e `instructions`. Os dados coletados são do `cpu_atom` (E-cores), que foi o núcleo utilizado pelo escalonador do SO para os processos de curta duração.

3.1. Entrada Pequena (N = 1000)

Os resultados para entrada pequena estão na Tabela 2.

Tabela 2. Resultados do perf – Entrada pequena (N=1000)

Algoritmo	Cache-Refs	Cache-Misses	Miss%	Cycles
Bubblesort	31.271	11.714	37,46%	7.428.140
Radixsort	22.357	7.800	34,89%	1.142.876
Quicksort	23.253	12.192	52,43%	1.603.600
Mergesort	20.452	9.941	48,61%	1.437.276

Tabela 3. Instruções e tempo – Entrada pequena (N=1000)

Algoritmo	Instructions	Tempo (s)
Bubblesort	18.205.892	0,002670
Radixsort	514.516	0,000782
Quicksort	1.113.146	0,001025
Mergesort	1.318.853	0,001111

3.2. Entrada Grande (N = 10000)

Os resultados para entrada grande estão nas Tabelas 4 e 5.

Tabela 4. Resultados do perf – Entrada grande (N=10000)

Algoritmo	Cache-Refs	Cache-Misses	Miss%	Cycles
Bubblesort	463.917	233.125	50,25%	327.101.734
Radixsort	27.153	11.358	41,83%	4.012.568
Quicksort	22.037	10.739	48,73%	4.786.270
Mergesort	22.351	9.479	42,41%	5.712.636

Tabela 5. Instruções e tempo – Entrada grande (N=10000)

Algoritmo	Instructions	Tempo (s)
Bubblesort	1.185.401.160	0,097168
Radixsort	7.252.696	0,001620
Quicksort	6.450.751	0,001987
Mergesort	9.000.140	0,002414

3.3. Análise Comparativa

Comparando as entradas pequena e grande:

- **Bubblesort:** apresentou a maior degradação. Cache-references cresceram 15x, cache-misses 20x e instruções 65x. A taxa de misses subiu de 37,46% para 50,25%, confirmando a ineficiência $O(n^2)$ para grandes entradas. O tempo saltou de 2,67 ms para 97,17 ms – aumento de 36x.

- **Radixsort:** manteve comportamento estável e foi o mais rápido na entrada grande (1,62 ms). Cache-references cresceram apenas 1,2x e misses permaneceram praticamente iguais. Isso reflete sua complexidade linear $O(nk)$ e padrão de acesso previsível.
- **Quicksort:** escalou bem, com crescimento moderado. Tempo de 1,02 ms para 1,99 ms (2x). A taxa de misses se manteve em 48-52%. Seu padrão in-place gera boa localidade, mas a recursão introduz algum overhead.
- **Mergesort:** apresentou excelente escalabilidade. Cache-misses praticamente não mudaram (9.941 para 9.479) entre $N=1000$ e $N=10000$ – o melhor resultado entre todos os algoritmos. Isso ocorre porque o merge acessa os arrays de forma sequencial, aproveitando a localidade espacial e o prefetcher de hardware. O tempo cresceu de 1,11 ms para 2,41 ms (2x), compatível com $O(n \log n)$.

4. Experimentos com Valgrind/Cachegrind – Item 5

4.1. Observação sobre o Ambiente

Durante a execução local do Valgrind 3.23.0 e 3.24.0 no Debian 13 com o processador Intel Core i5-13420H (arquitetura híbrida Raptor Lake), o Cachegrind gerou apenas estatísticas de referências de instruções (`I refs`), sem produzir as estatísticas de cache de dados (`D refs`, `D1 misses`, `LLd misses`). Este é um comportamento conhecido do Cachegrind em processadores com arquitetura híbrida P-core/E-core, onde o simulador de cache não consegue mapear corretamente os eventos de dados neste tipo de topologia de núcleos. O problema persistiu mesmo após reinstalação e uso do `taskset` para fixar a execução em um único core.

Por isso, os experimentos do item 5 foram realizados via Google Colab, que utiliza o mesmo Cachegrind em ambiente compatível (processador homogêneo).

4.2. Resultados com Parâmetros Variados

O algoritmo Torre de Hanoi recursivo com 20 discos foi executado com diferentes configurações de cache D1. Os resultados estão na Tabela 6.

Tabela 6. Resultados do Cachegrind – Torre de Hanoi (20 discos) com parâmetros variados

Size	Assoc.	Linha	D1 misses	LLd misses
2 KB	2-way	32 B	8.524	1.841
4 KB	2-way	32 B	5.778	1.841
8 KB	4-way	64 B	2.758	1.841
16 KB	8-way	64 B	2.265	1.841

O total de `D refs` foi constante em 15.775.872 em todos os testes, pois é determinado pelo algoritmo e não pela cache.

4.3. Análise

Os resultados mostram uma redução progressiva de `D1 misses` conforme o tamanho e a associatividade da cache aumentam:

- De 2 KB para 4 KB: misses reduzidos em 32% (8.524 → 5.778). A cache maior acomoda mais frames da pilha de recursão.
- De 4 KB para 8 KB com maior associatividade: misses reduzidos em 52% (5.778 → 2.758). O aumento da associatividade (2-way → 4-way) elimina conflitos de mapeamento entre frames com endereços que colidem no mesmo conjunto de cache.
- De 8 KB para 16 KB: misses reduzidos em 18% (2.758 → 2.265). O ganho marginal diminui porque a pilha ativa já cabe majoritariamente na cache de 8 KB.
- Os L1d misses permaneceram constantes em 1.841, mostrando que o que varia entre os testes é apenas a eficiência da cache L1 – o L3 permanece estável.

Isso demonstra os princípios de **localidade temporal** (frames acessados recentemente são acessados novamente) e o impacto da **associatividade** na redução de conflitos.

5. Otimização de Cache – Item 6: Hanoi Iterativo

5.1. Motivação

A versão recursiva da Torre de Hanoi cria um frame de ativação por chamada na call stack do sistema operacional. Os endereços dos frames são determinados pelo SO e podem estar espalhados na memória, causando cache misses por falta de localidade espacial entre frames consecutivos.

5.2. Solução: Pilha Explícita Contígua

A versão iterativa substitui a recursão por uma pilha manual alocada com `malloc()` em um único bloco contíguo de memória. Cada elemento é uma struct compacta de 16 bytes:

Listing 1. Struct da pilha manual – versão iterativa

```

1 typedef struct {
2     int  n;        // numero de discos
3     char from;     // pino de origem
4     char to;       // pino de destino
5     char aux;      // pino auxiliar
6     char phase;    // 0 = primeira chamada, 1 = apos mover disco maior
7 } Frame;          // 16 bytes por frame

```

Com linha de cache de 64 bytes e frames de 16 bytes, **4 frames cabem em uma única linha de cache**. Acessos consecutivos ao topo da pilha (`pilha[top]` e `pilha[top-1]`) estão na mesma linha ou em linhas adjacentes, explorando a localidade espacial.

5.3. Resultados Comparativos

Tabela 7. Comparação perf – Hanoi recursivo vs iterativo (20 discos)

Versão	Cache-Refs	Cache-Misses	Miss%	Cycles	Tempo (s)
Recursivo	35.675	12.731	35,69%	12.686.359	0,003462
Iterativo	8.917	3.667	41,12%	33.802.015	0,007950

5.4. Análise dos Resultados

- **Cache-references:** o iterativo gerou apenas 8.917 referências contra 35.675 do recursivo – redução de **75%**. A pilha contígua gera muito menos pressão sobre a cache.
- **Cache-misses:** o iterativo teve 3.667 misses contra 12.731 do recursivo – redução de **71%**. Confirma que a localidade espacial da pilha contígua funciona.
- **Cycles e tempo:** o iterativo usou mais ciclos (33,8M vs 12,7M) e foi mais lento (7,95 ms vs 3,46 ms). Isso ocorre porque a versão iterativa possui overhead de loop explícito, gerenciamento manual de estado (`phase`), cálculo do pino auxiliar faltante a cada iteração, e consequentemente muito mais instruções (87M vs 24M).
- **Conclusão:** a otimização de cache reduziu os misses em 71%, mas o custo extra de instruções superou o ganho. Em algoritmos que trabalham com arrays grandes (onde o ganho de cache é proporcional ao volume de dados), a otimização seria mais efetiva. Este experimento demonstra que otimização de cache deve ser avaliada junto com o overhead de código introduzido.

6. Conclusão

Este trabalho demonstrou como a hierarquia de cache influencia o desempenho de algoritmos. Os principais aprendizados foram:

1. O Bubblesort, com complexidade $O(n^2)$, degradou drasticamente com entradas grandes, apresentando aumento de 36x no tempo e 20x nos cache-misses ao passar de $N=1000$ para $N=10000$.
2. Radixsort, Quicksort e Mergesort escalaram bem ($O(n \log n)$ ou melhor), com crescimento de tempo de apenas 2x para 10x mais elementos. O Mergesort se destacou por manter os cache-misses quase constantes graças ao acesso sequencial durante o merge.
3. Aumentar o tamanho da cache e a associatividade reduz diretamente os cache misses, conforme demonstrado no Cachelgrind: de 8.524 misses com 2 KB para 2.265 misses com 16 KB na Torre de Hanoi.
4. Otimizações de cache (pilha contígua no Hanoi iterativo) reduzem misses em até 71%, mas o impacto no tempo real depende do equilíbrio com o overhead de instruções adicionais introduzido pela otimização.
5. O processador Intel Core i5-13420H, com arquitetura híbrida P-core/E-core Raptor Lake, apresentou incompatibilidade com o Cachelgrind para coleta de D refs localmente, exigindo o uso do Google Colab para os experimentos do item 5.

Referências

- GeeksForGeeks. *Bubble Sort Algorithm*. <https://www.geeksforgeeks.org/bubble-sort/>
- GeeksForGeeks. *Radix Sort*. <https://www.geeksforgeeks.org/radix-sort/>
- GeeksForGeeks. *QuickSort Algorithm*. <https://www.geeksforgeeks.org/quick-sort/>
- GeeksForGeeks. *Merge Sort Algorithm*. <https://www.geeksforgeeks.org/merge-sort/>

- GeeksForGeeks. *C Program for Tower of Hanoi*. <https://www.geeksforgeeks.org/c-program-for-tower-of-hanoi/>
- Intel. *Intel Core i5-13420H Processor Specifications*. <https://www.intel.com/content/www/us/en/products/sku/232173>
- Valgrind. *Cachegrind: a cache and branch-prediction profiler*. <https://valgrind.org/docs/manual/cg-manual.html>
- CPU-Z Validator. *Intel Core i5-13420H – Raptor Lake*. <https://valid.x86.fr/nwv17s>